WEEK 4 SECURITY REPORT

Advanced Threat Detection & Web Security Enhancements

NodeGoat Application

Student Name	Fatima Haq
Course	Cybersecurity
Project Title	Advanced Threat Detection and Web Security Enhancements in NodeGoat
Report Week	Week 4
Date	July 14, 2026
Tasks Completed	Task 1: Intrusion Detection & Monitoring Task 2: API Security Hardening Task 3: Security Headers & CSP

1. Objective

The objective of Week 4 was to strengthen the security of the NodeGoat application by implementing advanced security mechanisms, protecting API endpoints, monitoring suspicious activities, and configuring secure HTTP headers. This week focused on operational security — detecting threats in real-time, restricting access, and hardening the browser security surface.

Security Control	Package / Tool	Protection Against	Status
Login Monitoring	`winston`	Undetected brute-force	**Fixed**
Rate Limiting	`express-rate-limit`	Brute-force attacks	**Fixed**
CORS Configuration	`cors`	Cross-origin abuse	**Fixed**
API Key Authentication	`Custom middleware`	Unauthorized API access	**Fixed**
Helmet / CSP	`helmet`	XSS, clickjacking	**Fixed**
HSTS	`helmet`	Downgrade attacks	**Fixed**

Task 1 Intrusion Detection & Monitoring

Objective

To monitor suspicious login activities and record failed authentication attempts for security analysis and incident detection.

Implementation

Fail2Ban and OSSEC were considered for intrusion detection; however, both tools are Linux-native and are not supported in the Windows environment used for this project. As an alternative, Winston Logger was configured to provide real-time monitoring of failed authentication events.

Failed login attempts are now recorded with the following details:

- Username that was attempted
- IP address of the client
- Timestamp of the attempt
- Warning-level log classification

Implementation example in session.js:

```
// On invalid username:
logger.warn(`Failed login attempt (Invalid Username): ${userName} | IP:
${req.ip}`);

// On invalid password:
logger.warn(`Failed login attempt (Invalid Password): ${userName} | IP:
${req.ip}`);
```

Example entry recorded in security.log:

```
2025-01-15T09:44:12.301Z [WARN] Failed login attempt (Invalid Username):
hacker123 | IP: 127.0.0.1
```

```
2025-01-15T09:44:18.512Z [WARN] Failed login attempt (Invalid Password): admin | IP: 127.0.0.1
```

Note: Logging provides visibility but does not block attacks on its own. Rate limiting (Task 2.1) is the control that enforces actual blocking. Together they form a detect-and-block layered defense.

Testing

Several invalid username and password combinations were entered on the login page. Each failed attempt was captured and written to security.log in real time.

Result

Result: Real-time monitoring of failed login attempts was successfully implemented. Every failed authentication event is now logged with username, IP address, and timestamp.

Task 2 API Security Hardening

Task 2 covers three sub-tasks that together harden the application's API layer: rate limiting, CORS configuration, and API key authentication.

2.1 Rate Limiting

Objective

To protect the application against brute-force login attacks by limiting the number of requests a client can make within a defined time window.

Implementation

The express-rate-limit package was installed:

```
npm install express-rate-limit
```

The application was configured to allow only five requests per client within a fifteen-minute window:

```
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
    windowMs: 15 * 60 * 1000,   // 15 minutes
    max: 5,                       // max 5 requests per window
    message: "Too many requests. Please try again later."
});

app.use(limiter);
```

Parameter	Value
Maximum Requests	5
Time Window	15 Minutes
HTTP Response Code	429 Too Many Requests
Reset	After window expires

Testing

Multiple rapid requests were sent to the application. After exceeding the configured limit, the application blocked further requests and returned the rate-limit message.

Result

Result: Rate limiting successfully protected the application from brute-force attacks. Clients exceeding 5 requests in 15 minutes are blocked automatically.

2.2 CORS Configuration

Objective

To restrict which origins can send requests to the application, preventing cross-origin abuse from unauthorized websites.

Implementation

The cors package was installed:

```
npm install cors
```

CORS was configured to allow requests only from the trusted application origin:

```
const cors = require("cors");

app.use(cors({
    origin: "http://localhost:4000",
    methods: ["GET", "POST"],
    credentials: true
}));
```

Note: In production, replace http://localhost:4000 with the live application domain. Never set origin: '*' in a production environment — it disables all CORS protection.

Result

Result: Unauthorized origins are now restricted from accessing the application. Only requests from http://localhost:4000 are accepted.

2.3 API Key Authentication

Objective

To prevent unauthorized clients from accessing protected API endpoints by requiring a valid API key with every request.

Implementation

A custom API key authentication middleware was implemented:

```
const apiKeyAuth = (req, res, next) => {
    const apiKey = req.headers["x-api-key"];
    if (!apiKey || apiKey !== process.env.API_KEY) {
        return res.status(401).json({ error: "Unauthorized" });
    }
    next();
};
```

Unauthorized requests without a valid API key receive:

```
HTTP 401 Unauthorized
```

Note: The API key must be stored in an environment variable (process.env.API_KEY) and must never be hardcoded in source code. Hardcoded secrets can be extracted by anyone who reads the code.

Result

Result: API endpoints are protected against unauthorized access. All requests without a valid API key are rejected with a 401 Unauthorized response.

Task 3 Security Headers & Content Security Policy

Objective

To enhance browser security by applying secure HTTP headers that protect against Cross-Site Scripting (XSS), clickjacking, information disclosure, and insecure connections.

3.1 Helmet Configuration

Helmet middleware was configured to automatically apply a set of secure HTTP response headers:

```
const helmet = require("helmet");

app.use(helmet());
app.disable("x-powered-by");   // removes Express version fingerprint
```

Headers applied by Helmet include:

- X-Frame-Options: SAMEORIGIN — prevents clickjacking
- X-Content-Type-Options: nosniff — prevents MIME type sniffing
- X-DNS-Prefetch-Control — controls browser DNS prefetching
- Referrer-Policy — limits referrer information leakage

3.2 Content Security Policy (CSP)

A Content Security Policy was implemented to reduce the risk of Cross-Site Scripting attacks. CSP instructs the browser to only load scripts, styles, and other resources from trusted sources, blocking execution of injected or external malicious scripts.

```
app.use(helmet.contentSecurityPolicy({
    directives: {
        defaultSrc: ["'self'"],
        scriptSrc:  ["'self'"],
        styleSrc:   ["'self'"],
        imgSrc:     ["'self'", "data:"],
        objectSrc:  ["'none'"],
        frameAncestors: ["'none'"]
    }
}));
```

3.3 HTTP Strict Transport Security (HSTS)

HSTS headers were enabled to instruct browsers to always connect to the application over HTTPS, even if an HTTP link is clicked. This prevents protocol downgrade attacks and cookie hijacking over unencrypted connections.

```
app.use(helmet.hsts({
    maxAge: 31536000,         // 1 year in seconds
    includeSubDomains: true,
    preload: true
}));
```

Note: HSTS takes full effect only when the application is deployed over HTTPS in production. In the current local HTTP development environment the header is set in code and will activate upon HTTPS deployment.

Result

Result: The application now includes Helmet security headers, a Content Security Policy, and HSTS. Browser-level protections against XSS, clickjacking, MIME sniffing, and insecure connections are active.

4. Testing Summary

The following tests were successfully completed during Week 4:

#	Test Performed	Task Reference	Status
1	Failed Login Monitoring	Task 1	✔ Passed
2	Rate Limiting	Task 2.1	✔ Passed
3	CORS Restriction	Task 2.2	✔ Passed
4	API Key Authentication	Task 2.3	✔ Passed
5	Helmet Configuration	Task 3	✔ Passed
6	CSP Implementation	Task 3	✔ Passed
7	HSTS Configuration	Task 3	✔ Passed

5. Conclusion

Week 4 significantly improved the security posture of the NodeGoat application through the implementation of seven advanced security controls across three task areas.

The following protections were successfully implemented:

- ☑ Intrusion monitoring — failed login attempts logged with username, IP, and timestamp
- ☑ Rate limiting — brute-force attacks blocked after 5 requests per 15 minutes
- ☑ CORS restriction — cross-origin requests limited to the trusted application origin
- ☑ API key authentication — unauthorized clients rejected with HTTP 401
- ☑ Helmet headers — X-Frame-Options, X-Content-Type-Options, and others applied
- ☑ Content Security Policy — inline and external script execution restricted
- ☑ HSTS — browsers instructed to enforce HTTPS-only connections

These improvements reduce the application's exposure to brute-force attacks, unauthorized API access, cross-origin abuse, XSS escalation, and network interception. Combined with the authentication hardening from Week 2 and the penetration testing from Week 3, the NodeGoat application now has a comprehensive layered security posture covering monitoring, access control, transport security, and browser-level defenses.